Communication Protocols

Introduction:

Simplex

- One device transmits and other device receive.
- Communication is possible in one direction only.
- Ex:- ~~Ant~~ Radio



Half-Duplex

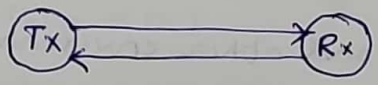
- two devices can send and receive from each others.
- two way communication but only one way at a time.
- Ex- Walkie-talkie.



Full duplex

- Communication is possible in both directions, both sides can transmit and receive at the same time.

Ex- Phone call.



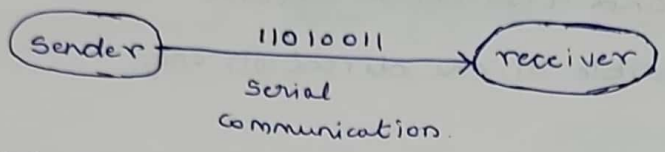
* Types of Communication.

1. Serial communication:

- The data will be sent (bit by bit), one bit at a time.
- Less number of cables required to transmit data, as it needs one channel or ~~cable~~ wire for transmitting (Low cost).

Scanned with CamScanner

- Sequentially transmission (slower)
- For correct data transmission, there has to be some form of synchronization between Tx and Rx.
- Used for all long-haul communication and most computer networks.



Ex- UART, I2C, SPI, USB, CAN, LIN ...

2. Parallel communication:

- More than bit can be transmitted at the same time over a link that consists of several parallel channels.
- Each bit is transmitted over a seperate channel, allowing for faster data transfer compared to serial communication.
- Parallel communication is typically used for short-distance transmissions due to certain limitations and challenges it faces.

(i) Clock skew -

- the clock signal (sent from the clock CKt) arrives at different components at different times.
- This happens because of wire-interconnect length, capacitive coupling, ~~mati~~ material imperfections and other environment factors.
- It reduces the speed of every link to the slowest of all of the links.

(ii) Cross talk -

- It occurs when the transmitted signal is adversely affected by another nearby signal.
- It happens when electromagnetic energy from one cable or channel leaves an imprint on adjacent cables, causing interference.

Scanned with CamScanner

features of parallel communication include

1. Mostly used for short-distance communication
2. It offers fast data transfer rates (high bandwidth)
3. Simplicity - It is relatively simple in terms of hardware implementation since it doesnot require serialization or deserialization components but it requires multiple channels or cables
4. Less susceptible to noise.

Ex- PCI Bus (Peripheral component interconnect).

3. Wireless communication.

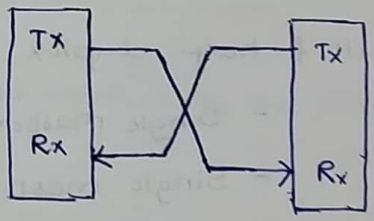
- This is done using IR (infrared) or RF (Radio frequency).

Ex - Bluetooth, ZigBee, WiFi, 2G, 3G ... etc.

* Types of Communication System.

1. Asynchronous:

- The transmission of the data don't use external clock.
- Synchronization is done using a fixed baud rate and using synchronization bits (start and stop bits)
- The baud rate defined by the lowest one of the devices.



- Byte oriented communication (data is a group "Bytes")
- Ex- UART

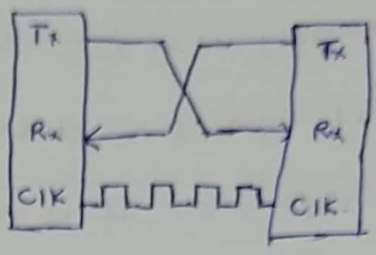
2. Synchronous:

- The transmission of the data uses external clock. [Common clk]
- transmission is synchronized over clk.

Scanned with CamScanner

- The transmitter sends the data with the clock
- Used at short distance
- More data to send
- Faster than Asynchronous communication

Ex- SPI, I2C, USART, USB.....



Serial peripheral interface (SPI)

- The master synchronous serial port (MSSP) module is a serial interface, useful for communicating with other peripherals or mμc devices.
- These peripheral may be serial EEPROMs, shift registers, ADC etc
- The MSSP module can operate in one of two modes
  - Serial peripheral interface (SPI)
  - Inter-integrated circuit (I2C)

* SPI : specifications.

→ Synchronous serial communication protocol.

→ Master slave topology (commonly single master multi slaves)

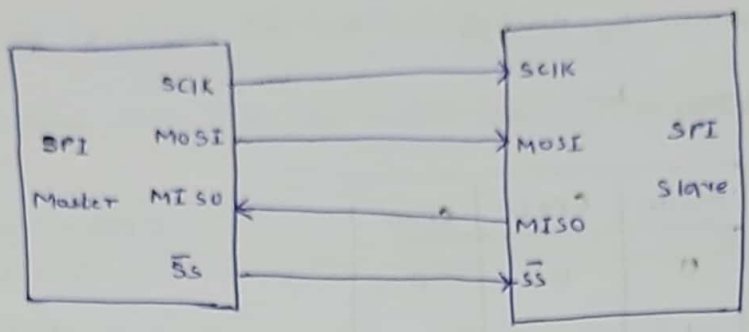
→ Used for short distances communications.

→ Full & half duplex modes.
  - Single master - Single slave.
  - Single master - Multi slaves
  - Multi master (depend on the MC ex. STM32407)

→ The master provides the communication clock (SCK)(CLK)

→ Requires (4 or 3 pins)
  - MOSI - Master output slave input. (From Master to slave)
  - MISO - Master Input slave Output (From slave to Master)
  - SCLK - Serial Clock (driven by master)
  - SS or CS - Slave or chip select.

Scanned with CamScanner



* For half duplex

- Data I/O - CIK - SS

↳ Same pin used for transmitting and receiving

→ The SS pin is used by master to select the slave that it wishes to communicate with.

- One SS pin is required per slave.
- So, if "n" slaves are connected to a master in an SPI bus configuration, "n" SS pins are required on the master.

→ For a bus configuration where 'n' slaves are connected to a Master (3+n) pins will be required on the master.

→ There is no standard set of speeds defined for the SPI protocol (depends on the Clock frequency → 10, 20, 50 Mbps) → High Speed

→ Higher throughput → Since there is no overhead added by the protocol such as addressing and flow control.

→ Lower power requirements than I2C protocol. Hence this is used in battery operated applications than I2C protocol.

→ Master Controlls all communications

→ No flow Control - master and slave must determined the communication speed.

→ GPIO - push-pull configuration (Advantage).

Scanned with CamScanner

→ Based on clock polarity and phase the SPI protocol divided into 4-modes.

Clock Idle is Low

Mode	CPOL	CPHA
0	0	0
1	0	1
2	1	0
3	1	1

IDLE = Low
- First leading edge
- second trailing edge

IDLE = high
- First Fallingedge
- Second Risingedge

→ Clock Idle is high.

CPOL = 0 ⊓↑ initialize clock from low to high

CPOL = 1 ↓⊔ initialize clock from high to Low.

CPHA = 0 ↑⊓ data exchanges at Leading edge

CPHA = 1 ⊓↓ data exchanges at trailing edge

~~SCK~~ ~~CPOL = 0~~

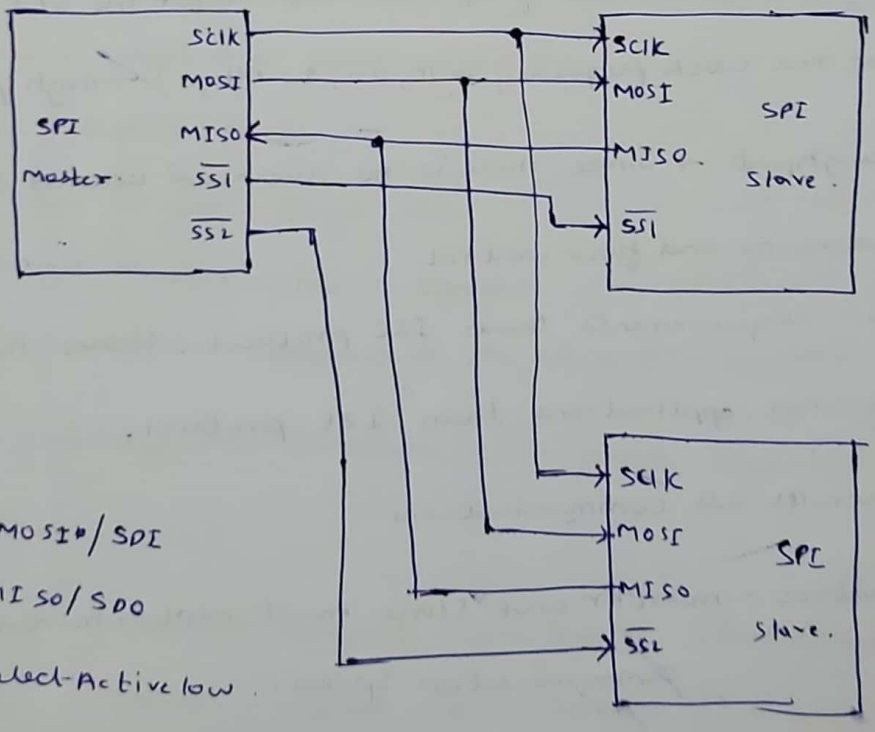
Clock polarity → 'Transmit' data.

Clock Phase → When to "Read or Sample" data.

Types of SPI interfacing:

1. Direct-Connection -

- Independent slave Configuration



* Full duplex
* 4-wires

MOSI ⟷ MOSI / SDI

MISO ⟵ MISO / SDO

$\overline{SS}$ → chip select-Active low.

Scanned with CamScanner

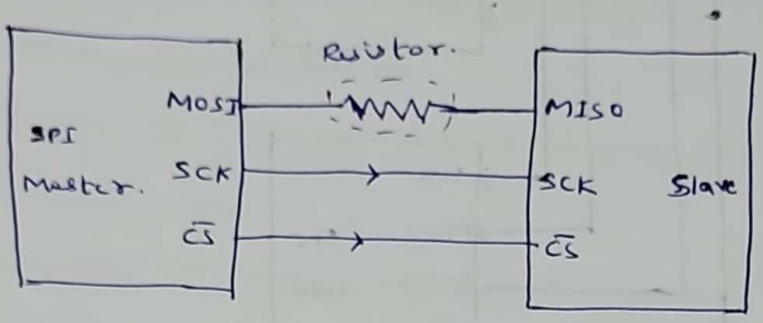
2. Quasi Bidirectional : (Half duplex mode)

→ Both master and slave use the same data line for transmitting and reception.

→ The transmission and reception done synchronously.

→ The Master and Slave needs to alternate their Roles Synchronously

→ It is common to add a serial resistor to prevent temporary short circuit connection.



3. Simplex Mode :

→ Unidirectional mode, only one data line is used.

→ MOSI → MOSI (or) MISO ← MISO.

→ SCK → SCK

→ $\overline{CS}$ → $\overline{CS}$

4. Daisy chain :

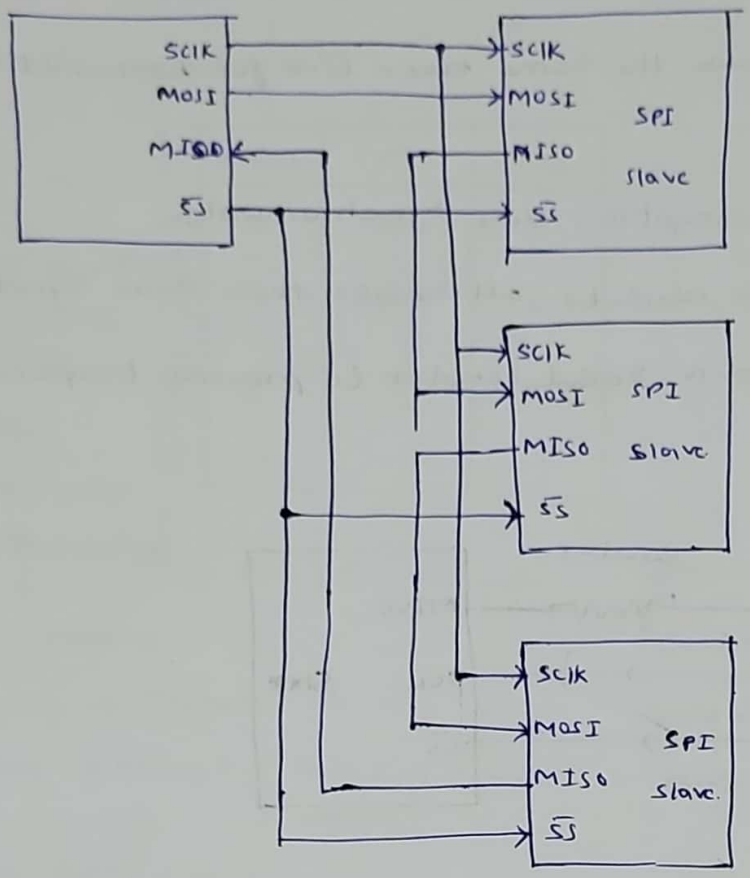
→ Master MOSI connected to Slave-1 MISO.

→ Slave-1 MOSI connected to Master MISO.

→ Master Clock line connected to all slave (SCK)

→ Master CS (only one CS from the master) connected to all Slaves CS.

Scanned with CamScanner



1st clock-

Data in from Master to slave.

2nd Clock-

Data out from slave 3 to Master

* Choosing the suitable slave in daisy chain :-

1. Determine the number of slaves devices connected in daisy chain and their respective addresses. Each slave device should have a unique address assigned to it.

2. Define a data package structure that includes the address information and the actual data to be transmitted to the slave device.

3. For transmitting data packet, each slave device will continously receive the data stream. When a slave device receives the address byte in the packet it compares the received address with its own pre-defined address.

4. If the received address matches, the slave device process the payload data.

Scanned with CamScanner

5. Multi Master topology. (MCU specific - STM32F4)

→ For example there are two MCU nodes capable of acting as master in the system.

→ All nodes, including the two MCU nodes, are set to slave mode, waiting for communication.

→ When a Node wants to communicate it switches itself to activate the master function, taking control of the two SPI bus.

- Responsible of clock generation and slave selection.
- Start communication session.

→ CS pin detect if there are potential bus collisions, indicating the start of communication.

→ After completing the communication the master node ~~remains~~ returns to the slave mode, allowing other node to take control of the bus or to end the communication.

MISO ⟷ MISO

MOSI ⟷ MOSI

SCK ⟷ SCK

GPIO ⟶ CS

CS ⟵ GPIO.

Advantages -

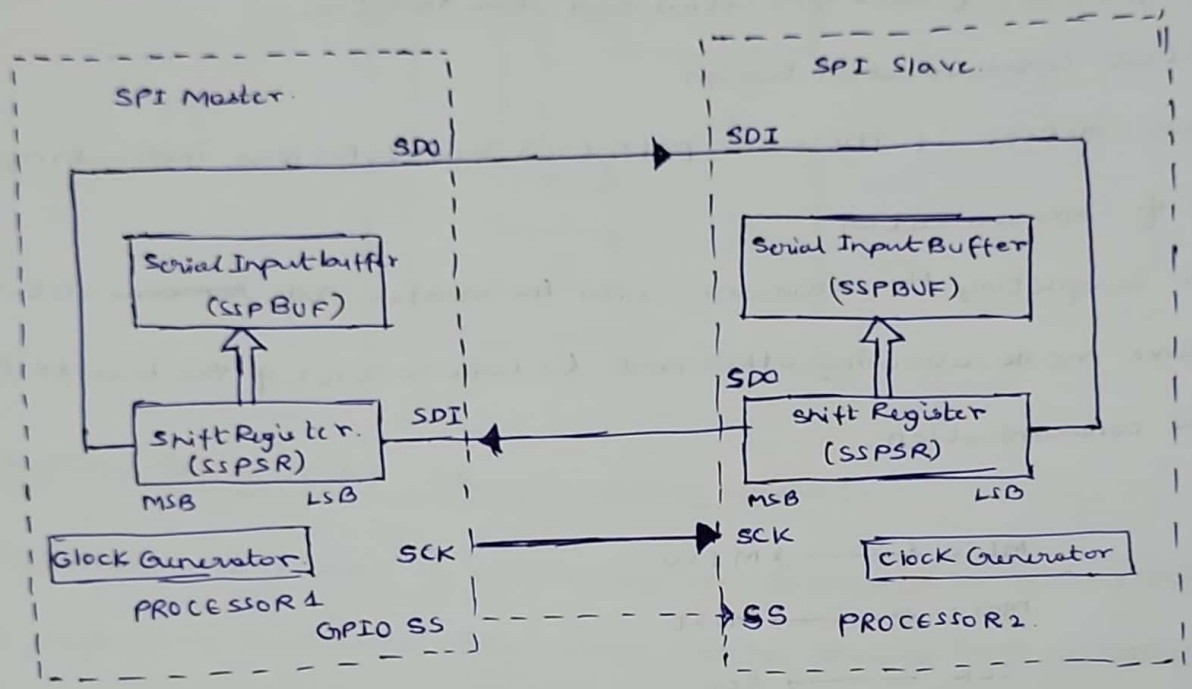
- No start and stop bits, so the data can be ~~stored~~ streamed continuously without interruption.
- No complicated slave addressing system like I2C.
- Higher data transfer rate than I2C (almost twice as fast)
- Seperate MISO and MOSI lines, so data can be sent and received at the same time.

Scanned with CamScanner

Disadvantages -

- Uses four wires (I2C and UART uses two).
- No acknowledgement that data has been successfully received (I2C has this)
- No form of error checking like the parity bit in UART.
- Only allows for a single master.

How SPI works



→ SPI consists of two shift registers, one in the master and the other in the slave side.

→ Also there is a clock generator in the master side that generates the clock for the shift registers.

→ Shift registers are 8-bit long, so after 8 clock pulses, the contents of the shift registers are interchanged.

→ When the master wants to send a byte of data, it places the byte in its shift register and after 8 clock pulses the data will be transmitted to the other shift register.

Scanned with CamScanner

→ When the master wants to receive a byte of data, the slave side should place the byte in its shift register, and after 8 clock pulses the data will be received by the master shift register.

→ It must be noted that SPI is full duplex, meaning that it sends and receives data at the same time.

→ SSPSR is the shift register for the SPI module. It shifts data in and out of a device. The data travels in a loop to the next shift register. The data is shifted out the SDO pin of ~~the~~ one device and onto the SDI pin of the other.

→ A clock (SCK) is generated by the master device. It controls when and how quickly data is exchanged between the two devices.

→ SS, allows a master device to control when a particular slave is being addressed. The allows the possibility of having more than one slave and simplifies the communications. When the SS signal goes low at a slave device, only that slave is accessed by SPI.

→ Once a byte of data has been exchanged between the two devices, it is copied to the SSPBUF register. The SSPBUF is then read by the user software. If any data is to be exchanged, this register is written to by your program. Writing two the SSPBUF will transfer the contents ~~to the~~ of SSPBUF to the SSPSR.

SCK [8] Master ←	0 1 0 0 0 0 1 0	0 1 0 0 0 0 0 1 ← Slave
SCK [7] master ←	0 0 1 0 0 0 0 1	0 0 1 0 0 0 0 0 ← Slave
SCK [6] Master ←	0 1 0 1 0 0 0 0	1 0 0 1 0 0 0 0 ← Slave
SCK [5] Master ←	0 0 1 0 1 0 0 0	0 1 0 0 1 0 0 0 ← Slave
SCK [4] Master ←	0 0 0 1 0 1 0 0	0 0 1 0 0 1 0 0 ← Slave
SCK [3] Master ←	0 0 0 0 1 0 1 0	0 0 0 1 0 0 1 0 ← Slave
SCK [2] Master ←	0 0 0 0 0 1 0 1	0 0 0 0 1 0 0 1 ← Slave
SCK [1] Master ←	1 0 0 0 0 0 1 0	1 0 0 0 0 1 0 0 ← Slave

Scanned with CamScanner

Inter Integrated Circuit

* I2C characteristics :

→ Serial protocol - where data is sent bit by bit over a single wire known as serial data line (SDA).

→ Synchronous protocol - Each bit is synchronized with a clock signal provided by the serial clock (SCL) line.

→ Half duplex - Bidirectional - which means that data can be transmitted in both directions but not simultaneously. The same I2C bus is used for transmitting and receiving data.

→ Byte oriented - The I2C protocol operates on a byte level, meaning that data is sent and received in chunks of 8-bits (1-byte) at a time.

→ Two-wire Bus - The I2C bus consists of two lines: SDA and SCL. these two wires are shared among multiple devices connected to the bus.

→ Multi-Master - Multi-slave -

- There is only a master device that controls the communication at a time, and one or more slave devices that responds to the masters commands.
- Each devices on the bus is assigned a unique software address allowing the master to communicate with individual slaves.
- The Master device generates the clock signal used for synchronizing transfer on the bus.
- In case two or more masters try to access the bus simultaneously collision detection and bus arbitration mechanisms are used to resolve conflicts and prevent data corruption.

Scanned with CamScanner

→ Support Multiple Speeds.

1. Bi-directional bus specification

   (a) Standard Mode (SM) upto : 100 Kbit/sec.

   (b) Fast Mode (FM) upto : 400 Kbit/sec.

   (c) Fast Mode Plus (FM+) upto : 1 Mbit/sec

   (d) High Speed Mode (HSM) upto : 3.4 Mbit/sec

2. Unidirectional bus Specifications.

   (a) Ultrafast Mode (U-FM) upto : 5 Mbit/sec.

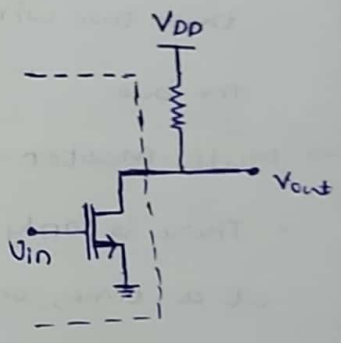
   - Master to slave or slave to master, only one direction.
   - Doesn't support multi master option.

* a defining characteritics of I2C is that every device on the bus must connect to both clock signal (SCL) and data signal (SDA) via open-drain or (open collector) output drivers.

⇒ Here are three important implications of open-drain configuration:

(a) The Signal always default to logic high

- The devices can determine that the bus is available for new transmission by observing that both SCL and SDA have been logic high for certain amount of time.



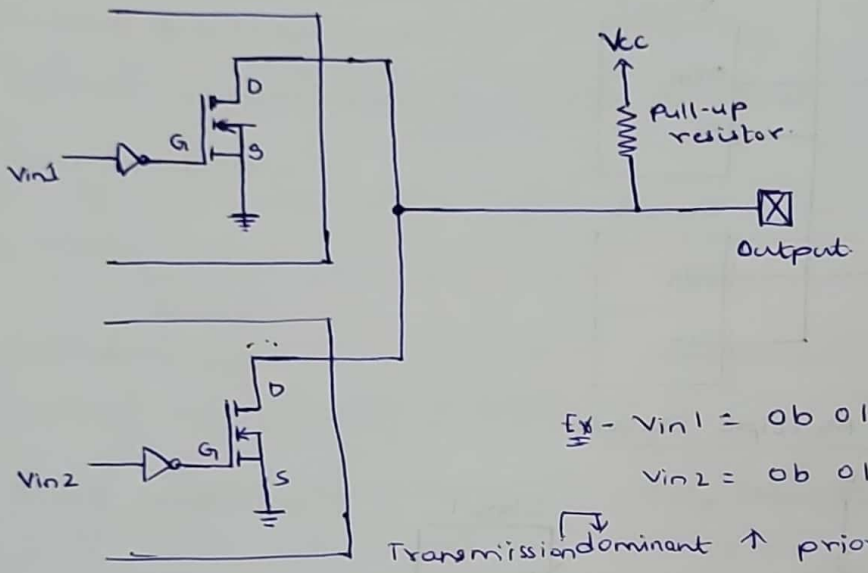
(b) Any device on the bus can safely drive the signals to logic "0", even if another device is trying to drive them high - "And Wiring"

- The 'And wiring' concept emerges when multiple devices are connected using 'open drain' outputs. When any device actively pulls the line Low, it effectively brings the line low for all other devices connected to it. However, when no device is actively pulling the Line low, the pull-up resistors ensure that the line is pulled high.

Scanned with CamScanner

- To handle simultaneous transmission attempts, the I2C protocol incorporates additional mechanisms such as "Bus arbitration and collision detection", to ensure that only one device is actively transmitting on the bus at any given time, and preventing data corruption
- During the arbitration, master monitors the bus and compare their intend logic level with the observed level. If a master detects mismatch it realizes that a higher priority master is currently transmitting and gives up control of the bus

Ex-



Vin1	Vin2	Output
Low	Low	Low
Low	high	Low
high	Low	low
high	high	high.

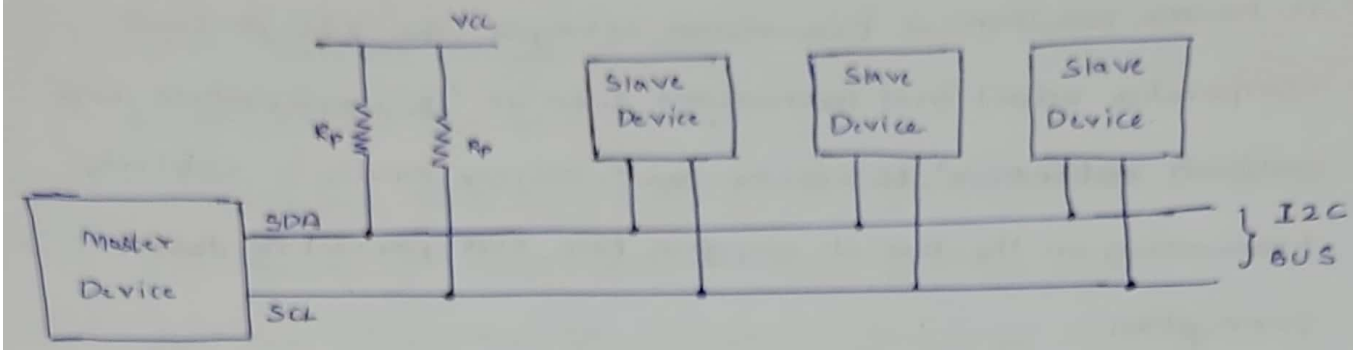
Ex- Vin1 = ob 0101**0**100 → 0 : dominant bit
Vin2 = ob 0101**1**100 → 1 : Recessive.

Transmission dominant ↑ priority, Recessive ↓ priority.

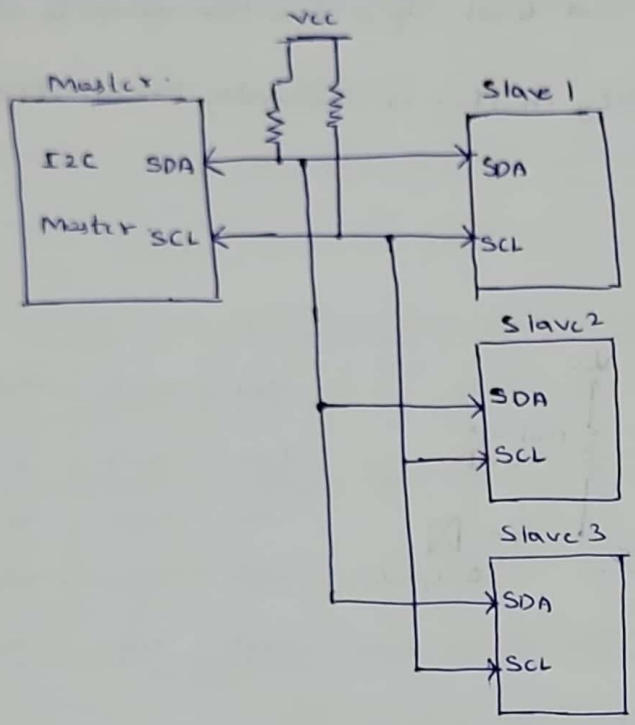
(C) Devices with different supply voltages can coexist on the same bus as long as the lower voltage devices will not be damaged by the higher voltage.

- We can do this by pulling up the SCL and SDA to the lowest voltage using pull-up resistor. for example (3.3V)
- OR we can do this by using Bi-directional logic level converter.

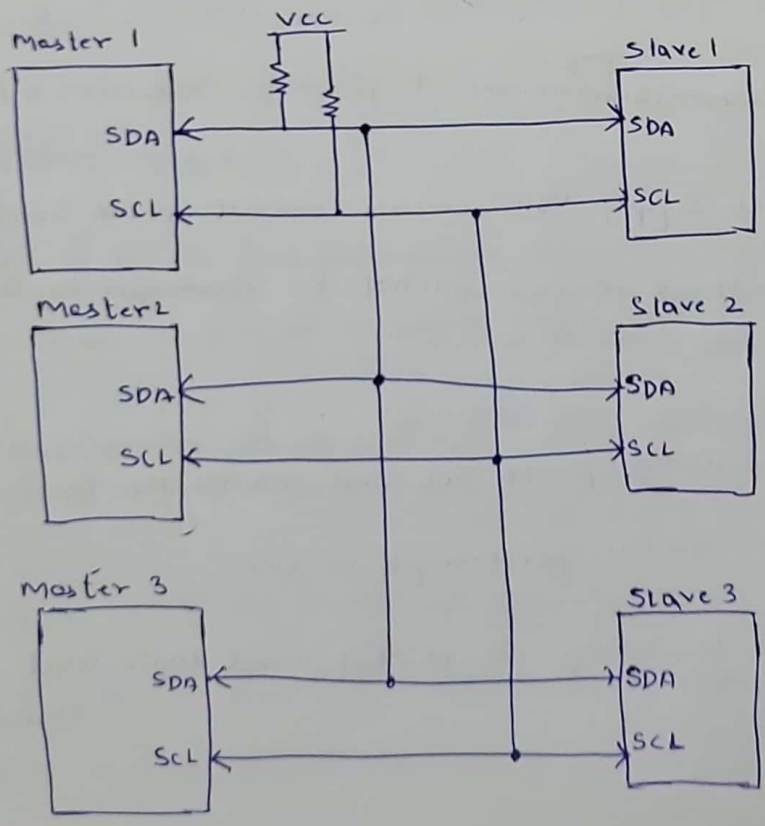
Scanned with CamScanner



* Single Master Multi-slave.



* Multi Masters with Multiple Slaves.



Scanned with CamScanner

* I2C Frame format :

1. Start and Stop conditions -

→ all transactions begin with a start and are terminated by a stop condition.

→ A high to low transition on the SDA line, while SCL is high defines a start condition.

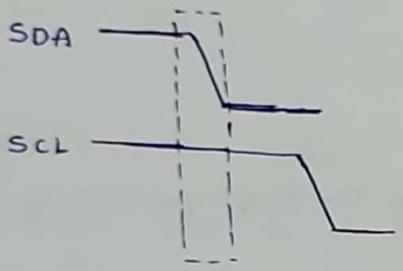
→ A low to high transition on the SDA line, while SCL is high, defines a stop condition.

→ start and stop conditions are always generated by the master.

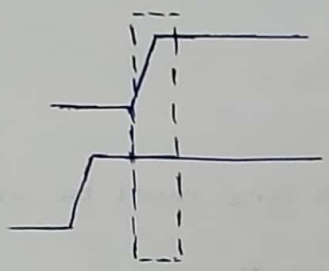
→ The Bus is considered to be busy after the start condition.

→ The Bus stays busy if a repeated start condition is generated instead of a stop condition.

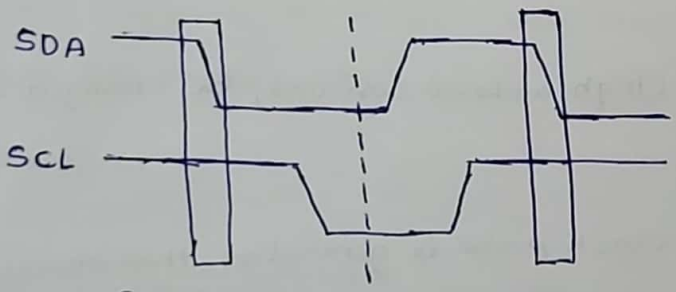
→ The Bus is considered to be free again at a certain time after the stop condition.



Start condition 'S'



Stop Condition 'P'



Start condition 'S'

Repeated Start condition. 'Sr'

→ detection of start and stop conditions by devices connected to the bus is easy if they incorporate the necessary interfacing hardware.

→ However, microcontrollers with no such interface have to sample the SDA line at least twice per clock period to sense the transistor.

Scanned with CamScanner

2. Byte format -

→ Every word transmitted on the SDA line must be 8-bits long (byte-oriented)

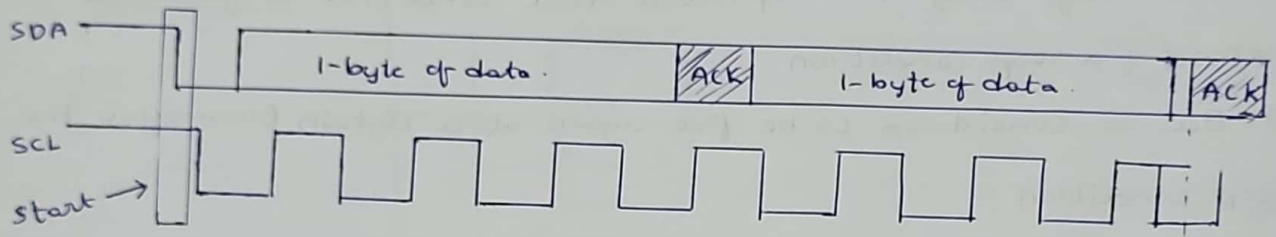
→ The number of bytes that can be transmitted per transfer is unrestricted.

→ Data is transfered with the Most Significant bit (MSB) first.

→ Each byte must be followed by an Acknowledgement (ACK) bit.

→ If slave needs to perform other tasks before receiving or transmitting another byte of data, it can hold the clock line (SCL) Low, this action pauses the data transfer and forces the master device to wait.

→ When the slave is ready for another byte, it releases the clock line and data transfer resumes.

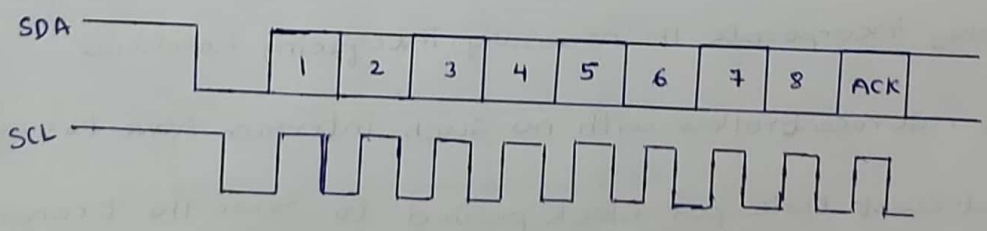


3. Data Validity -

→ the data on the SDA line must be stable during the high period of the clock, any changes to the data line data during this period can lead to data corruption.

→ The state of the data line (High or Low) can only be changed when the clock signal on the SCL is low.

→ For each bit of data transfer, one clock pulse is generated, this means inorder to transfer one byte of data (8-bits), the master generates 9 clock pulses. the extra clock pulse is used for the ACK bit that is sent by the receiving device.



Scanned with CamScanner

4. Acknowledge (ACK) and Not Acknowledge (NACK) -

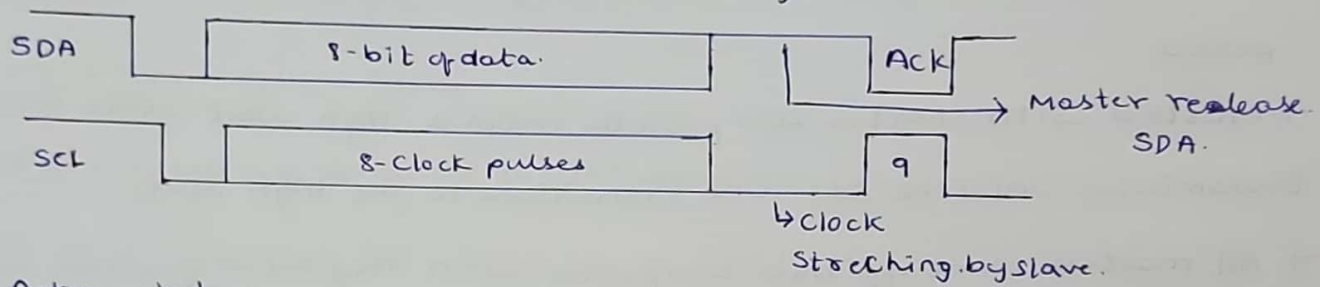
→ The ACK takes place after every byte.

→ The ACK bit allows the receiver to signal the transmitter that the byte was successfully received.

→ The master generates all clock pulses, including the ACK ninth clock pulse.

→ The transmitter releases the SDA line during the ACK clock pulse so the receiver can pull the SDA line low and it remains stable low during the high period of this clock pulse.

→ If a receiver device needs to perform other tasks before receiving or transmitting another byte of data, it can hold the SCL line low to force the master into a wait state (clock streching).



* Not Acknowledge:

→ When SDA remains high during the ninth clock pulse, this is defined as Not Acknowledge Signal (NACK).

→ When this happens, the master can then generate either a stop condition to abort the transfer or send a repeated start condition to start a new transfer.

5. Clock Synchronization -

→ In an I2C system with mutiple masters, clock synchronization and arbitration mechanisms are necessary to determine the bus master and complete its transmission.

→ Single master systems do not require these mechanisms as there are only one master device.

Scanned with CamScanner

→ Clock synchronization is acheived through the wired-AND connection of I2C interfaces to the SCL line.

→ When the SCL line transitions from High to Low, it triggers the master to start counting their low period.

→ Each master has its own internal clock for timing data transmission.

→ Once a masters clock reaches a low state, it holds the SCL line until the clock transition back to high.

→ this keeps the SCL line for the master's low period, allowing data transmission.

→ If another master's clock is still within its Low period when one master tries to transitions ~~from~~ the SCL line from Low to High, the SCL Line may not immediately change state.

→ In this case, the SCL line is held low by the master with longest Low period.

→ Masters with shorter Low periods enter a High wait-state and pause transmission until the SCL line transitions to the High State.

→ All masters count off their Low period, when they have finished counting, the clock line is released and transitions to the High State.

→ At this point, there is no difference between the master's clocks and the State of the SCL Line.

→ All masters start counting their High periods simultaneously.

→ The ~~fas~~ first master to complete its high period pulls the SCL line back Low.

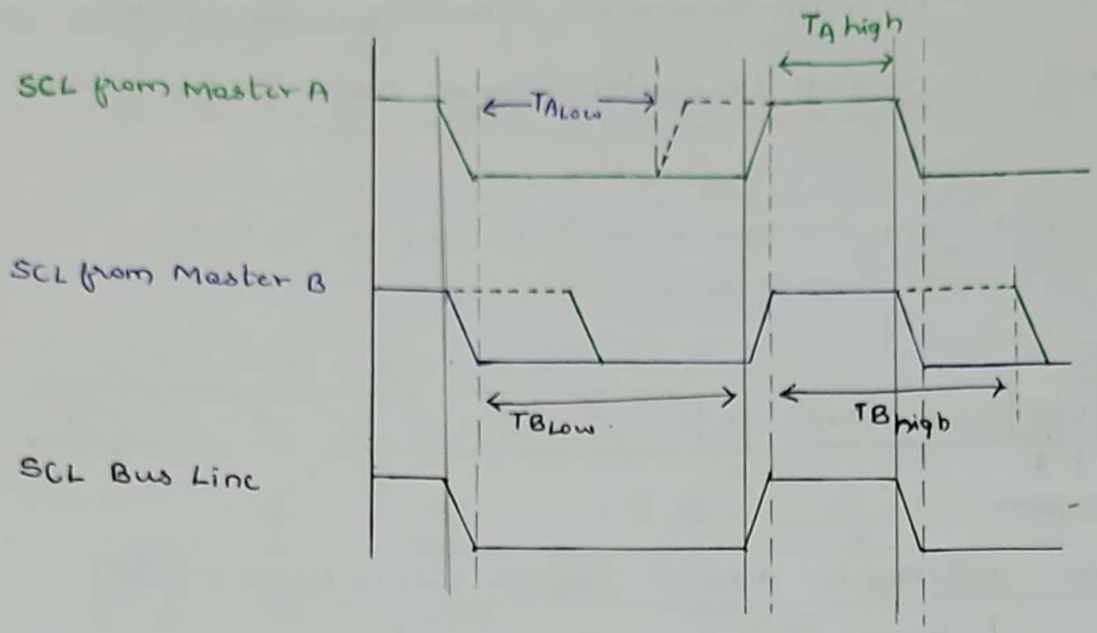
→ This process generates a Synchronized SCL clock.

→ The Low period of the SCL clock is determined by the master with the longest clock low period.

→ The high period of the SCL clock is determined by the master with the Shortest

→ Starting from the second clock, the SCL clock will be Synchronized.

Scanned with CamScanner



Point 1 : Masters starts counting Low period.

Point 2 : Masters start counting High period.

The Slave address and R/$\bar{W}$ bit :

1. 7-bit slave address Mode.-

→ Data transfers follows the format shown:-

1- Start Condition (S) : initiates the data transmission.

2 - Sending a 7-bit slave address : The master sends the 7-bit address of the target slave device.

3-- 8th bit (R/$\bar{W}$) : Following the 7-bit address, an additional bit, the R/$\bar{W}$ bit is transmitted.

0 - Master needs to send data to the slave (Write)

1 - Master needs to receive data from the slave (Read).

4. Data transfer termination : Every data transfer concludes with a stop condition (P).

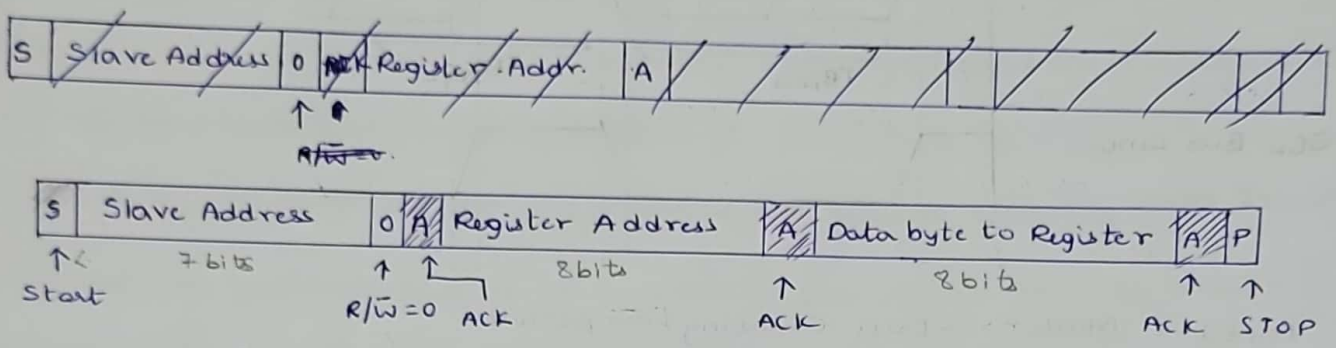
5. Optional repeated start condition (Sr) : if the master wishes to continue communicating on the bus with another slave without ending the current session, it can generate a repeated start condition (Sr) and address another slave without proceeding it with a stop condition.

Scanned with CamScanner

1. Master-transmitter to slave-receiver : (Unchanged direction).

→ In this format, the master-transmitter sends data to the slave-receiver.

→ The Slave-receiver acknowledges each byte sent by the master.



▨ Slave controls SDA Line ☐ Master controls SDA Line

2. Master-Receiver to Slave transmitter : (Unchanged direction).

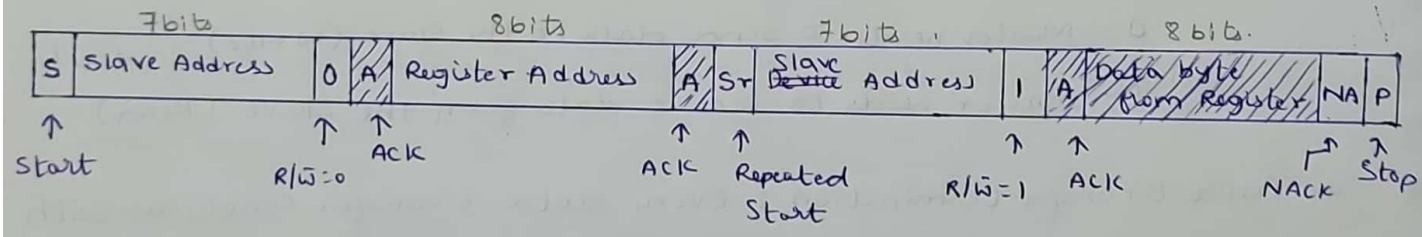
→ Master reads from the Slave immediately after the first byte.

→ at the moment of the first acknowledgment, the master-transmitter becomes a master receiver and the slave-receiver becomes a slave transmitter.

→ The first acknowledge is still generated by the slave.

→ The master generates subsequent acknowledges.

→ The Stop condition is generated by the master, which sends a not-ack (Ā) just ~~after~~ before the stop condition (do not need a read data anymore)



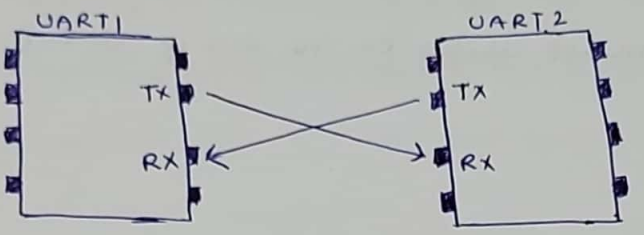
☐ Master controls SDA Line.

▨ Slave controls SDA Line.

Scanned with CamScanner

* UART protocol -

- UART stands for Universal Asynchronous Receiver/Transmitter.
- It's not a communication protocol like SPI and I2C, but a physical circuit in a microcontroller, or a stand-alone IC.
- A UART's main purpose is to transmit and receive serial data.



- UART's transmit data asynchronously, which means there is no clock signal to synchronize the output of bits from the transmitting.
- UART has only uses two wires to transmit data between devices.
- Both UART's must operate at the same baud rate. i.e. Maximum speed upto 115200 baud and usually 9600 baud is used.
- It supports only one master and one slave to it.

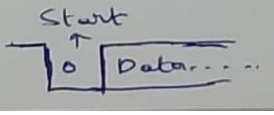
~~How UART Works~~

Data Frame Format.

- In UART, the mode of transmission is in the form of packet "Frame".
- A packet frame consists of a start-bit, data-bits, ~~and~~ parity bit and Stop bits.

Start-bit (1-bit)	Data Frame 5 to 9 data bits	Parity-bits 0/1 bit.	Stop-bits. 1 or 2 bits

Start bit - To Signal the start of the data packet, the Sender drives the data transmission Line from high to Low Voltage for 1-bit duration.



Scanned with CamScanner

Data Frame - The data field contains the actual data being transferred.

(i) Data field size : [5:8 bits] + parity bit.

(ii) Data field size : [9 bits] + no parity bit.

→ In most cases the data is sent with the least Significant bit (LSB).

Ex- A → ASCII code = 65 → 0 1 0 0 0 0 0 1 (MSB → LSB).

→ The Sender and receiver must align to the size of data field "must be the same number of bits".

Parity bit -

even parity
- '1' bits odd → parity bit = 1
- '1' bits even → parity bit = 0.

odd parity
- '1' bits odd → parity bit = 0
- '1' bits even → parity bit = 1

→ This bit is ~~used~~ added to check that the number is received correctly. because bits can be changed by electromagnetic radiation, mismatched baudrates or long distance transfers.

Stop bit -

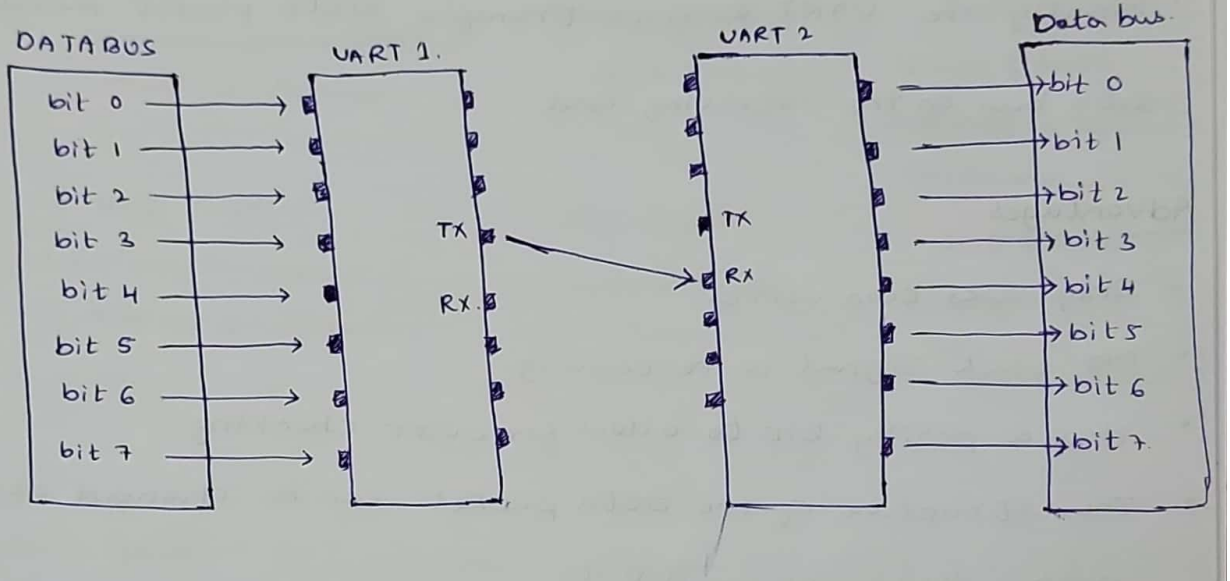
→ To signal the end of the data packet, the Sender drives the data transmission line from Low voltage for 1 bit or 2 bits or 1.5 bits duration.

Scanned with CamScanner

Working of UART

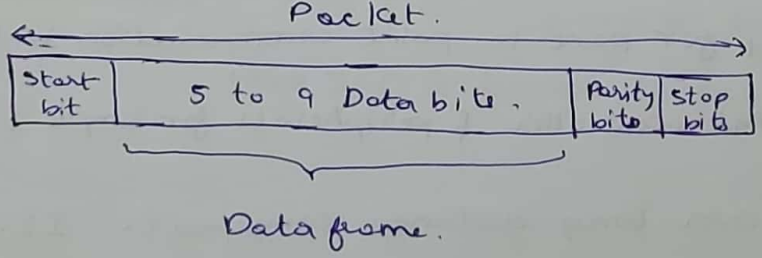
Step 1 - Data bus send data to the UART

- The UART that is going to transmit data receives data from a data bus.
- The data bus is used to send data to the UART by another device like a CPU, memory or microcontroller.
- Data is transferred from the data bus to the transmitting UART in parallel form.



Step 2 - UART creates data packets.

- After the transmitting UART gets the parallel data from the data bus.
- Then, UART creating the data packet by adding start bit, a parity bit and a stop bit.
- Each packet contains 1 start bit, 5 to 9 data bits (depending on the UART), an optional parity bit, and 1 or 2 stop bits.



Scanned with CamScanner

Step-3 : UART Transmitter send data to UART receiver.

- UART1 transmitter (TX pin) send this packet serially to UART2 Receiver (RX pin).
- The receiving UART reads the data packet bit by bit at its RX pin.

Step-4: UART Receiver received data & send it parallel.

- The receiving UART then converts the data back into parallel form and removes the start bit, parity bit and stop bits.
- Finally, the UART Receiver transfers data packet parallel to the data bus on the receiving end.

Advantages

- Only uses two wires.
- No clock signal is necessary
- Has a parity bit to allow for error checking.
- The structure of the data packet can be changed as long as both sides are set up for it.
- Well documented and widely used method.

Disadvantages

- The size of the data frame is limited to maximum of 9 bits.
- Doesn't support multiple slave or multiple master system.

Applications

- UART is widely used for point-to-point communication in various fields. Ex. Connection b/w Microcontroller & peripherals for simple & direct data exchange.
- UART communicates over long distances compared to I2C and SPI

Scanned with CamScanner

I2C vs SPI vs UART.

Characteristics.	I2C	SPI	UART
Speed.	Average (upto few Mbits/sec)	High (several Mbits/sec to a few tens of Mbits/sec)	Moderate (lower than that of SPI)
Complexity	Moderate (2 wires, management of several slaves)	Moderate to high. (4-Wire, full duplex)	Moderate. (2 wires.)
Use Case.	Ideal short distance communications with multiple devices.	Perfect for fast data transfer	Suitable for simple & long distance & serial communications.
Duplex	Half duplex (two way communication. but not simultaneous)	Full Duplex. (Simultaneous two-way communication)	Full duplex. (simultaneous. two-way communication)

Tip -

↳ Communication Speed - SPI for speed, UART for flexibility & I2C for less speed demanding configurations.

↳ Circuit design - I2C for efficient Space management with multiple devices. SPI for performance for larger designs & UART for Simplicity and Versatality.

↳ Distance & communication Environment - UART is Robust over long distances while I2C is better suited to short distances.

↳ Duplex Requirements - SPI & UART provides full-duplex capabilities, while I2C is limited to half duplex.

Scanned with CamScanner